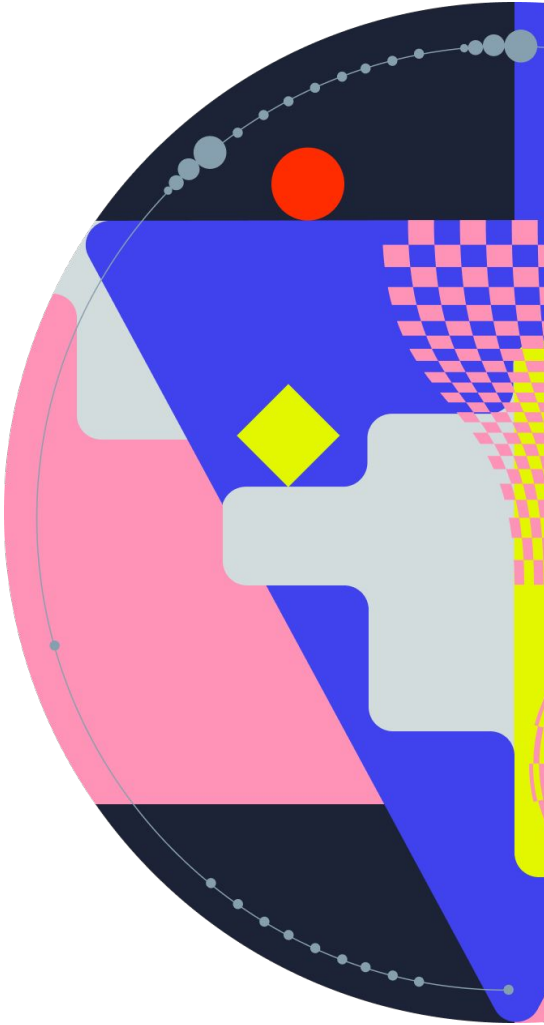
Школа анализа данных

Lecture 4

Agent Evaluation: From Metrics to Managed Quality

Sergey Kuptsov
Alice Agent Platform Advocate

Introduction



Part 1
Why agent eval is a separate discipline

What is Evaluation?

input task → system output → scoring logic → metric

- **Systematic, repeatable process** — not a one-time manual check
- **Automated**: runnable in CI, no real users required
- **Gives a number**: progress is measurable, regressions are detectable

As systems grew more complex, eval had to evolve with them

How Evaluation Evolved

Generation	What the system does	What we evaluate	Typical eval
Single-turn LLM	1 prompt → 1 answer	Answer quality	BLEU, ROUGE, exact match
Multi-turn LLM	Dialog with memory, several turns	Coherence, context, instruction following	Dialog metrics, human eval
Agentic systems	reasoning → action → observation loop, tools, env side-effects	Full trajectory + outcome + environment state	Graders: code + LLM-judge + human

What is an AI Agent?

Reasoning → Action → Observation → repeat

- **Calls tools**: APIs, databases, search, code execution
- **Modifies environment state** — actions have real-world consequences
- **Iterates until task is complete** (or gives up)

Key insight: we must evaluate the whole chain, not just the final text

Why Evaluate Agents?

- **Reliability & Safety** — catch bugs before production, not after
- **Understand Behavior** — why did the agent make that decision?
- **Track Progress & Regressions** — new model / prompt / tool → did it improve?
- **Iteration Speed** — evals turn dev into a managed process; without them: guessing
- **Objective Communication** — 'pass@1 dropped 8% on edge-cases' vs 'it seems worse'

Why Agent Eval is Hard

- **Compound errors** — mistake at step 3/10 propagates, corrupts all subsequent steps
- **Multiple valid paths** — different action sequences can reach the same correct outcome
- **Stochasticity** — same prompt → different reasoning chains; classical assertEqual fails
- **Hidden failures** — wrong tool, wrong param, but final text looks acceptable
- **Environment side-effects** — agent says 'done', but did the DB actually change?

Agentic Eval — Full Chain, Not Just the Answer

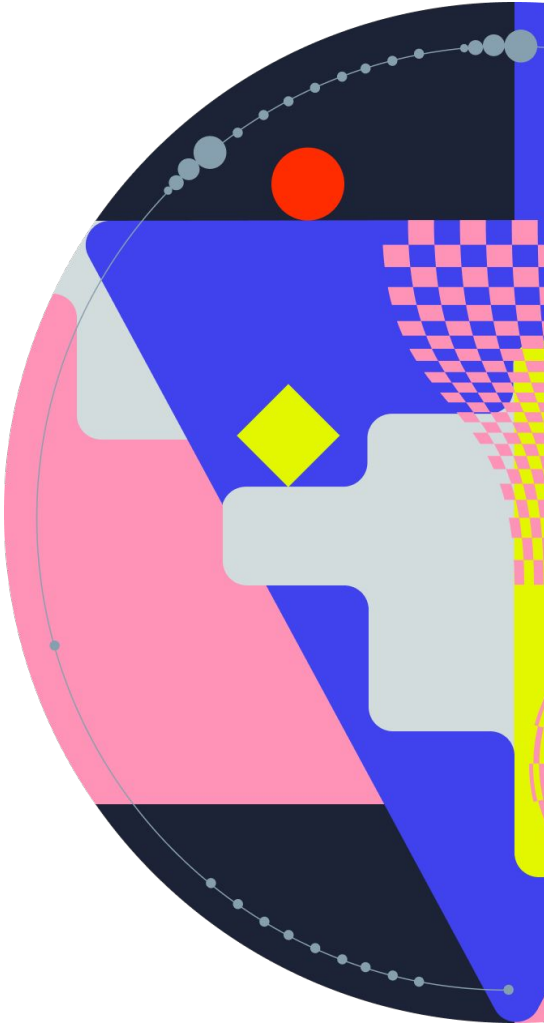
Full Trace
What steps were taken · What actions executed · How decisions were made

Technical Effects
Which API called · What parameters passed · What external system returned

Logical Effects
Did the agent understand context · Respect constraints · Complete the task

Eval of an agent = test of the entire reasoning-to-action chain, not just the final text

Terminology

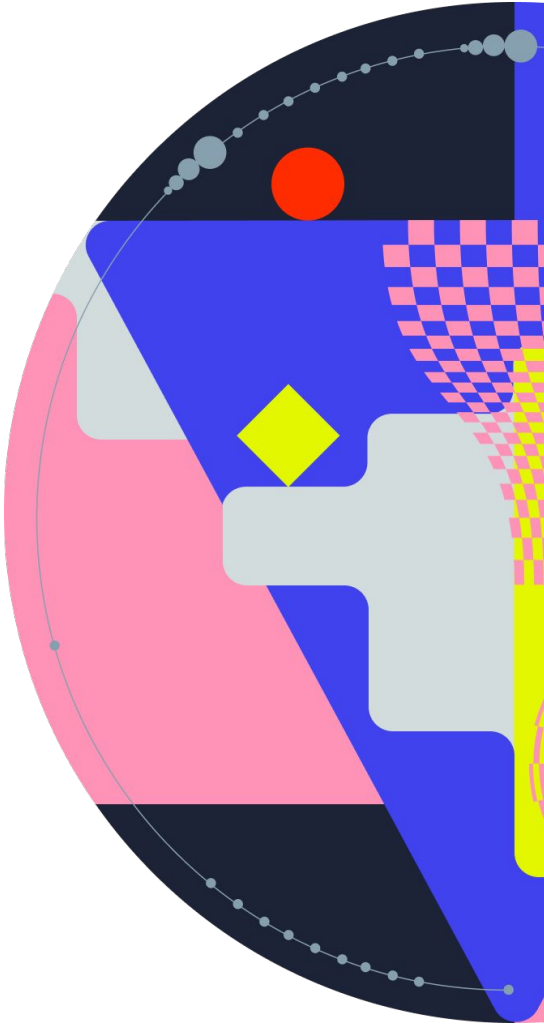


Part 2
Shared vocabulary

Key Terms

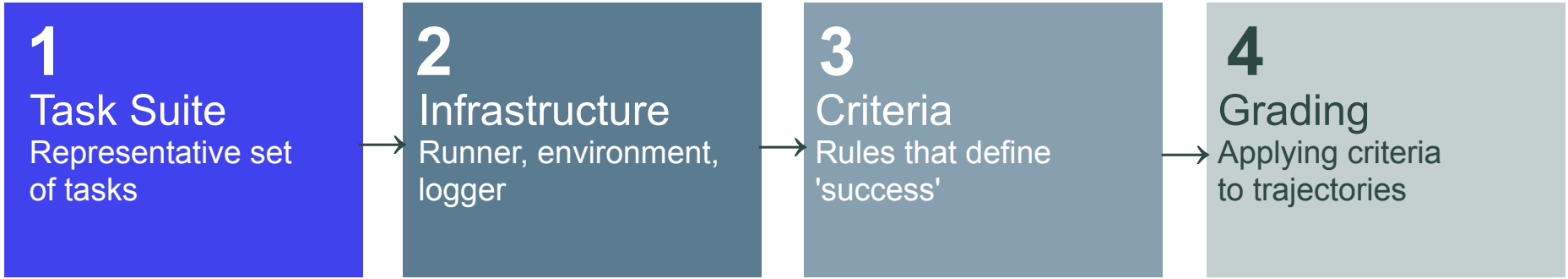
Evaluation Suite	Collection of tasks measuring specific agent abilities
Task / Test Case	One test: user query + env state + success criteria
Trial	One execution of a task. Multiple trials per task (stochasticity!)
Trajectory	Full record: outputs, tool calls, reasoning, observations
Outcome	Final environment state after trial — the real DB change, not the text
Grader	Logic that evaluates one aspect of the agent's work
Eval Harness	Infrastructure that runs evals: orchestrates tasks, captures results

Eval Pipeline Anatomy



Part 3
4 components — each is necessary

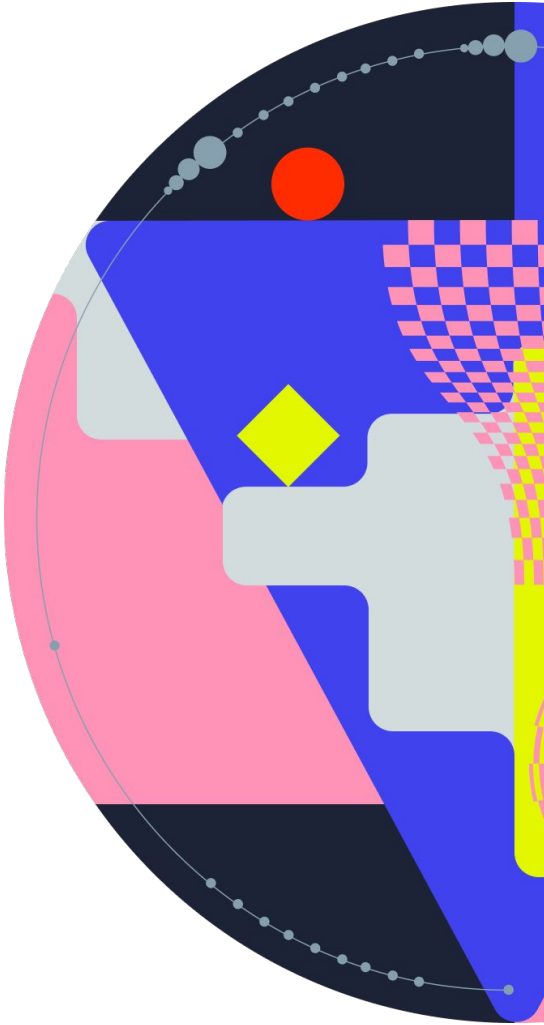
The 4-Component Eval Pipeline



Feedback loop: grading results feed back into all earlier stages

Weakness in any one component devalues all the others

Task Suite



Part 4
Pipeline component 1 of 4
The most valuable artifact

Building the Task Suite — Start Small

20–50 tasks from real problems is enough to start

- v0 — 5–10 tasks from manual analysis of what the agent should do
- v1 — run agent, find errors, add edge-cases and failure modes
- v2 — add categories, balance, LLM-generated tasks for gaps
- v3 — lock as regression suite (runs on every PR)
- v∞ — add tasks from bug tracker and user feedback after launch

Key insight: A well-constructed benchmark dataset is the foundation — all other evaluation components (criteria, graders, monitoring) depend on it

Sources for Tasks

- **Manual analysis of user needs** — always, from day one, what we want from agent
- **Production logs** — best distribution representation (once deployed)
- **Bug tracker / support** — capture real failure modes after launch
- **Public benchmarks** — τ-bench, SWE-bench, ToolBench (for calibration vs SOTA)
- **LLM generation** — features × scenarios × personas matrix, for scale

Rule: everything your team verifies manually → convert to a formal test

Scaling the Suite with LLM Generation

Generate tasks across 3 dimensions — then validate every generated task manually

Features

Agent functionalities

→ Search · Change · Cancel · Info

Scenarios

Interaction situations

→ Edge-case · No results · Ambiguous · Error

Personas

User types

→ Novice · Frustrated · Imprecise

Example generation prompt:

Generate an airline user query with:
- Feature: change booking
- Scenario: incorrect data (booking not found)
- Persona: frustrated, uses short phrases

Format: query text only, 1–2 sentences.

⚠️ Always validate manually

- Is the task unambiguous?
- Is it actually solvable?
- Does it make domain sense?

Tag every task:

- difficulty: easy / medium / hard
- category: info / change / cancel
- edge_case · needs_confirmation

Task = Query + Environment State

A task is NOT just a text prompt

1. **User query** — what the user wants
2. **Initial environment state** — DB, APIs, business policies in effect
3. **Expected goal state** — how DB should look after correct execution

- Agent says "Your flight is changed" — did the DB actually update?
- Grade on state change, NOT on response text — core principle of serious agentic evals

Task Structure Example

Airline agent — change booking to cheaper flight

```
user_query:  'Reschedule ABC123 to tomorrow, only if cheaper'

initial_environment:
  flights:
    SU2454_0220: { date: 2026-02-20, price: 25000 }
    SU2454_0221: { date: 2026-02-21, price: 24000 }
  bookings:
    ABC123: { flight_key: SU2454_0220, status: confirmed }
  policies:
    - 'Change is free if new flight is cheaper'
    - 'Require user confirmation before changing'

expected_goal_state:
  bookings:
    ABC123: { flight_key: SU2454_0221, status: changed }
```

Grader compares final DB state to expected_goal_state

No text analysis needed — pure objective check

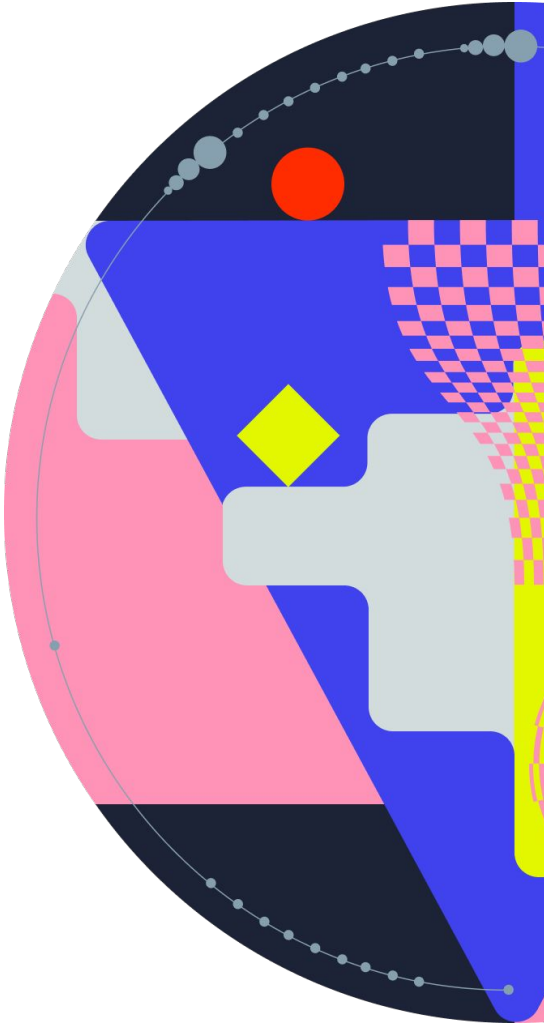
Policies are part of the environment, not just the prompt

Tools are declared — agent can only use listed tools

Properties of a Good Task Suite

- **Unambiguous** — two experts must agree on pass/fail
- **Environment state** — objective outcome check, not text analysis
- **Class balance** — don't overfit on 'easy' categories
- **Reference solution** — proves task is solvable, validates graders
- **Bidirectional** — test when agent SHOULD and SHOULD NOT act
- **Reproducible** — runs in CI; no dependency on live external APIs
- **Tagged** — by difficulty, type, skill (enables regression analysis)

Eval Infrastructure



Part 5
Pipeline component 2 of 4
Turning checks into engineering

Principle Zero: Same Agent Code

The agent in eval MUST be the same agent as in production

- Same code · Same prompts · Same tools — no 'simplified eval version'
- Prompts and tool descriptions come from one source of truth
- Only difference: test environment instead of production, test DB instead of real DB

- If eval agent behaves differently from production — eval conclusions don't transfer

Infrastructure Components

Runner

Runs agent on full suite
Concurrency · Timeouts · Retry

Environment

Isolated env per trial
Clean DB state on every run

Logger / Tracer

Full trajectory captured
Reasoning · Tool calls · Observations

Iron User

LLM-based user simulator
Required for all multi-turn dialog tasks

Storage

All runs · Trajectories · Metrics
Searchable · Versioned

Comparator

Metric diff across versions
Regression detection & visualization

Environment: Real vs Mock

◀ *more control · faster · cheaper* *more realistic · more honest* ▶

Full Mock	Hybrid	Production API
in-memory DB fake APIs	mock external APIs + real DB	all real calls
When: Early dev · CI unit tests for tools	When: Main eval mode best balance	When: Final validation pre-release only
⚠️ Mock drift — diverges from real API over time	⚠️ Keep test data fresh and realistic	⚠️ Slow · expensive · external service dependencies

Even in mock mode — data must be realistic: varied prices, nulls, edge-cases, sold-out seats

Policies are part of the environment — version them alongside test data

Iron User — Simulated User for Dialog Tasks

Without Iron User, dialog tasks cannot complete

- **Fixed goal** — knows what it 'wants', stays in scenario
- **Naturalistic** — short answers, doesn't reveal everything upfront
- **Does NOT hallucinate** data not in the instruction

Iron User — Problems to Know

- **Non-deterministic** — Iron User is also an LLM; its answers vary between runs
- **Can hallucinate** — may invent data not in scenario despite instructions
- **Too cooperative** — real users are vague, impatient; simulator inflates scores
- **Loop risk** — agent error → simulator repeats → infinite loop

- Fix: adaptive prompt, strict max_steps, correct API design (tools return enough info)
- Iron User is necessary but not sufficient — periodic real-user validation remains important

Iron User — Simulation Strategies

τ-bench implements 4 strategies with increasing reliability — each builds on the previous

LLM

How:
LLM generates a response directly from the scenario instruction

Why:
Baseline variant. Fast. Prone to hallucinating data not in the instruction.

ReAct

How:
LLM first reasons (Thought:), then generates the response (Response:)

Why:
More consistent behavior. Reasoning step keeps simulator aligned with scenario.

Verify

How:
After generating, a 2nd LLM call checks quality. If bad → regenerate (up to 3 times)

Why:
Guards against simulator hallucinations. Quality gate before response is sent to agent.

Reflection

How:
On failed verification: LLM reflects on the error, then generates an improved response

Why:
Most reliable. Explicit self-correction loop. Used in highest-quality τ-bench evals.

Key rule in system prompt: "Do not hallucinate information not in the instruction"

Agent asks for booking number not in scenario → simulator says "I don't remember", not invents one

Minimum Requirements for Your Own Harness

- **Runs on shared VM / CI** — not on a dev laptop
- **Retry** on API timeouts, graceful error handling
- **Auto-export results** (traces + metrics) to storage after every run
- **Parallel task execution** — serial execution of 100 tasks × 5 trials = hours
- **Version comparison** — at minimum a metric diff in CSV
- **Isolation** — clean environment state per trial (no leftover artifacts)

Eval Libraries & Platforms (2026)

Libraries

- DeepEval — pytest-style, 40+ metrics, CI
- RAGAS — reference-free RAG eval
- Arize Phoenix
- Braintrust AutoEvals — ready judge prompts
- inspect-ai — UK AISI, 95+ benchmarks

Platforms

- LangSmith — trajectory eval, framework-agnostic
- W&B Weave — scorers, leaderboards, human review
- OpenAI Platform — full harness, trace grading
- Braintrust — harness + AutoEvals + dashboard
- n8n Evaluations — no-code workflow builder

LangSmith — What It Looks Like

Experiment view

LangChain Team > Datasets & Experiments > Email Extraction > Comparing 1 Experiments

Edit resource tags 1 ENTERPRISE

Playground - 2025-02-04 03:56:04

Compact Full Diff JSON Heat Map Columns + Compare

Input	Reference Output	Output	Hallucination	Helpful	Prose	Latency	Status	Tokens
Consider all ways to give to... #38d4	{"tone":"positive","topic":"Donation and S...	ai: The email is a heartfelt ap... 1/3	1.00 µ	1.00 µ	0.50 µ	14.48s	SUCCESS 3	437.333
||||| OCTOBER 2023, VOL... #4eb6	{"tone":"positive","topic":"Princeton Univ...	ai: This email is a newsletter ... 1/3	1.00	0.00 µ	0.50 µ	14.48s	SUCCESS 3	5,428
USA Finance Solution offer... #5066	{"tone":"positive","topic":"Business fundi...	ai: This email is a promotion... 1/3	1.00 µ	1.00 µ	0.50 µ	14.48s	SUCCESS 3	356.333
Hello Jacob, For many small... #55cf	{"tone":"positive","topic":"Business finan...	ai: This email is a marketing ... 1/3	1.00 µ	1.00 µ	0.50 µ	14.48s	SUCCESS 3	585.667
| Auroras and Adventure A... #65de	{"tone":"positive","topic":"Travel and Tour...	ai: This email is a promotion... 1/3	1.00 µ	1.00 µ	0.50 µ	14.47s	SUCCESS 3	432.667
Solve the RedWordle We kn... #6a7d	{"tone":"positive","topic":"RedWordle Puz...	ai: The email is an engaging,... 1/3	1.00 µ	0.00 µ	0.50 µ	14.47s	SUCCESS 3	339.333
Your exclusive retreat at Th... #7bf0	{"tone":"positive","topic":"Exclusive retre...	ai: This email serves as a pr... 1/3	0.67 µ	1.00 µ	0.50 µ	14.47s	SUCCESS 3	726
Many of you aren't just runn... #7fad	{"tone":"positive","topic":"Funding solutio...	ai: This email is a promotion... 1/3	1.00 µ	1.00 µ	0.50 µ	14.47s	SUCCESS 3	515.667
#### Where sustainability ... #8412	{"tone":"positive","topic":"Promotion of U...	ai: This email is a promotion... 1/3	1.00 µ	1.00 µ	0.50 µ	14.47s	SUCCESS 3	308
Hi! I just announced that I w... #936c	{"tone":"positive","topic":"Campaign anno...	ai: This email is a campaign ... 1/3	1.00 µ	1.00 µ	0.50 µ	14.47s	SUCCESS 3	672
Hi there, | --- We've update... #c23c	{"tone":"positive","topic":"Update of Crun...	ai: The email is a formal com... 1/3	1.00 µ	1.00 µ	0.50 µ	12.98s	SUCCESS 3	364
--- | Costco --- ANSWER & ... #c4fa	{"tone":"positive","topic":"Invitation to par...	ai: This email appears to be ... 1/3	1.00 µ	0.33 µ	0.50 µ	12.98s	SUCCESS 3	578.333

LangSmith — Version Comparison

LangChain Blogs > Datasets & Experiments > Tweet Writing Task-backtestin... > Comparing 2 Experiments

Add resource tags | BETA USER

Comparing Experiments

Diff | Display

2 experiments compared by is_grounded against a Baseline of prod-baseline-gpt-3.5-turbo-0... where a higher score indicates better performance

Input		prod-baseline-gpt-3.5-turbo-0470 (Baseline)		candidate-gpt-4o-10ba8697 (0 ↑ 2 ↓)		Compare
human: Alan turing's ear...	#2779	ai: 🧠✨ Alan Turing's early childhood was filled with cur...	1.00	ai: 🌟 Alan Turing's early years were marked by incredi...	1.00	
human: Economic impac...	#cd3f	ai: The European Union brings 🌍 impactful benefits like...	1.00	ai: 🇪🇺🌍 The EU's economic landscape in 2023 is one of...	1.00	
human: Underrated phil...	#a745	ai: 🔍 Did you know there are many underrated philosop...	1.00	ai: 🧠✨ Ever heard of Abū Bakr Muhammad ibn Zakariy...	0.00	
human: History of the Ro...	#ac21	ai: 🎟️ Located in San Francisco's Mission District, the hi...	1.00	ai: 🎟️✨ The Roxie Theater in San Francisco, founded in ...	1.00	
human: ELI5: gravitation...	#d2f3	ai: 🌌 Exploring the cosmos with gravitational waves! Im...	1.00	ai: Imagine space is like a giant cosmic trampoline 🌌. G...	1.00	
human: The arguments f...	#a295	ai: 🏛️ The case against a written constitution in a parlia...	1.00	ai: A parliamentary system speeds up legislation 🚀 an...	0.00	
human: Pivotal moment...	#3da0	ai: 🎶 Dive into history with these iconic music moments...	1.00	ai: 🎸 From Jimi Hendrix setting his guitar ablaze at Mon...	1.00	
human: Big ideas in prog...	#5d61	ai: 💡 Explore new programming languages with over 10...	1.00	ai: 🚀 Exciting times in programming! In 2023, Rust is m...	1.00	
human: Big questions in...	#3a09	ai: 🌿 Exploring the big questions in biology uncovers hi...	1.00	ai: 🌿 What are the big questions in biology for 2023? 🤔...	1.00	
human: The relationship...	#00a2	ai: The relationship between math and reality is a deep ...	1.00	ai: 🔎 Ever wondered how math shapes our reality? 📐 F...	1.00	
human: What makes so...	#4935	ai: 🎭 What makes someone funny? Humor often links t...	1.00	ai: What makes someone funny? 🤔 Often, it's a mix of ...	1.00	

OpenAI Evals — What It Looks Like

Distillation Test / shikhar

Playground | Dashboard | Docs | API reference

DASHBOARD

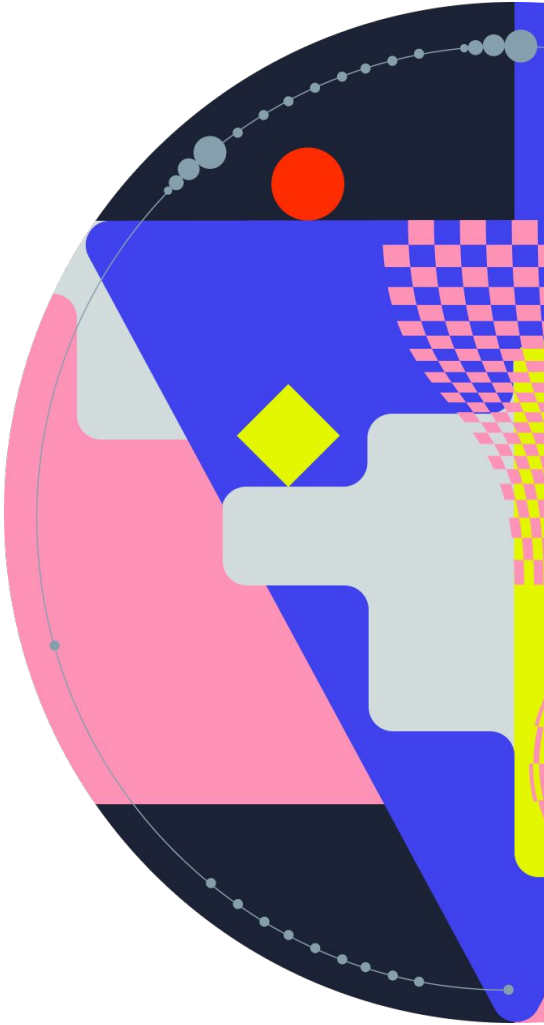
- Prompts
- Logs
- Traces
- Assistants
- Batches
- Datasets
- Evaluations
- Fine-tuning
- Storage
- Usage
- API keys

sentiment_extraction_eval | Report | Data | + Add run

Run gpt-4.1-sentiment | Sentiment Grader

Item			Sample View details		Testing Criteria
text	channel	language	Input	Output	Sentiment Grader: Result
It's okay – not great but not bad either.	app_review	en	System: You are a helpful assistant. User: It's okay – not great but not bad either.	Assistant: {"sentiment":"neutral","confidence":0.95,"emotions":["indifference","acceptance"]}	Pass: 4.5
I love this product!	twitter	en	System: You are a helpful assistant. User: I love this product!	Assistant: {"sentiment":"positive","confidence":0.98,"emotions":["joy","satisfaction"]}	Pass: 5
This is the worst experience I've ever had.	support_ticket	en	System: You are a helpful assistant. User: This is the worst experience I've ever had.	Assistant: {"sentiment":"negative","confidence":0.99,"emotions":["anger","frustration","disappointment"]}	Pass: 5
No estoy seguro de lo que pienso sobre este producto.	facebook	es	System: You are a helpful assistant. User: No estoy seguro de lo que pienso sobre este producto.	Assistant: {"sentiment":"neutral","confidence":0.95,"emotions":["indecisión"]}	Pass: 5
总体来说，我对这款产品很满意。	wechat	zh	System: You are a helpful assistant. User: 总体来说，我对这款产品很满意。	Assistant: {"sentiment":"positive","confidence":0.98,"emotions":["satisfaction","contentment"]}	Pass: 5

Criteria & Metrics



Part 6
Pipeline component 3 of 4
Defining success

What Makes a Good Criterion?

Start with 2–3 criteria — teams that start with 10 end up measuring nothing

✓	**Aligned**	Corresponds to the agent's goal — not a proxy metric
✓	**Operationalizable**	Simple to verify — automatable, two experts must agree on pass/fail
✓	**Human-readable**	Colleagues understand what it means without explanation
✓	**Discriminative**	On failure: immediately clear what exactly to fix
✓	**Stable**	Independent of subjective interpretation — doesn't drift between runs

If two independent experts disagree on pass/fail — the criterion needs reworking

Two Groups of Criteria

Output-based (Formal)

- Agent returned response in expected format
- Correct tool called with correct parameters
- Environment state changed as expected (booking in DB)
- No runtime errors or exceptions

Implemented as: assert · unit tests · state checks

Trajectory-based (Behavioral)

- Is the action chain logically consistent?
- Was the task goal maintained throughout?
- Were there unnecessary steps (inefficiency)?
- Are decisions justified in the reasoning?
- Did the agent lose context mid-trajectory?

Implemented as: LLM judge · count-based · reference compare

Both groups are needed — outcome-only grading misses process failures and policy violations

Universal Criterias

Start with 2–3 criteria — avoid metric drowning

Criterion	Question to answer	Type	Example Grader
Usefulness	Did the agent solve the user's task?	Output-based	State check · binary pass/fail
Correctness	Errors, exceptions, invalid tool calls?	Output-based	Assert · static analysis
Groundedness	All facts confirmed by tool data?	Trajectory	LLM judge · reference compare
Efficiency	Minimum steps, no redundant tool calls?	Trajectory	Count-based · LLM judge
Stability	Does success repeat on re-run?	Statistical	pass@k · pass^k
Safety	No toxicity, data leaks, policy violations?	Output / Traj	Rule-check · classifier

Stability Metrics: pass@k and pass^k

pass@k — AT LEAST ONE of k trials succeeds

$$\text{pass@k} = 1 - \frac{C(n-c, k)}{C(n, k)}$$

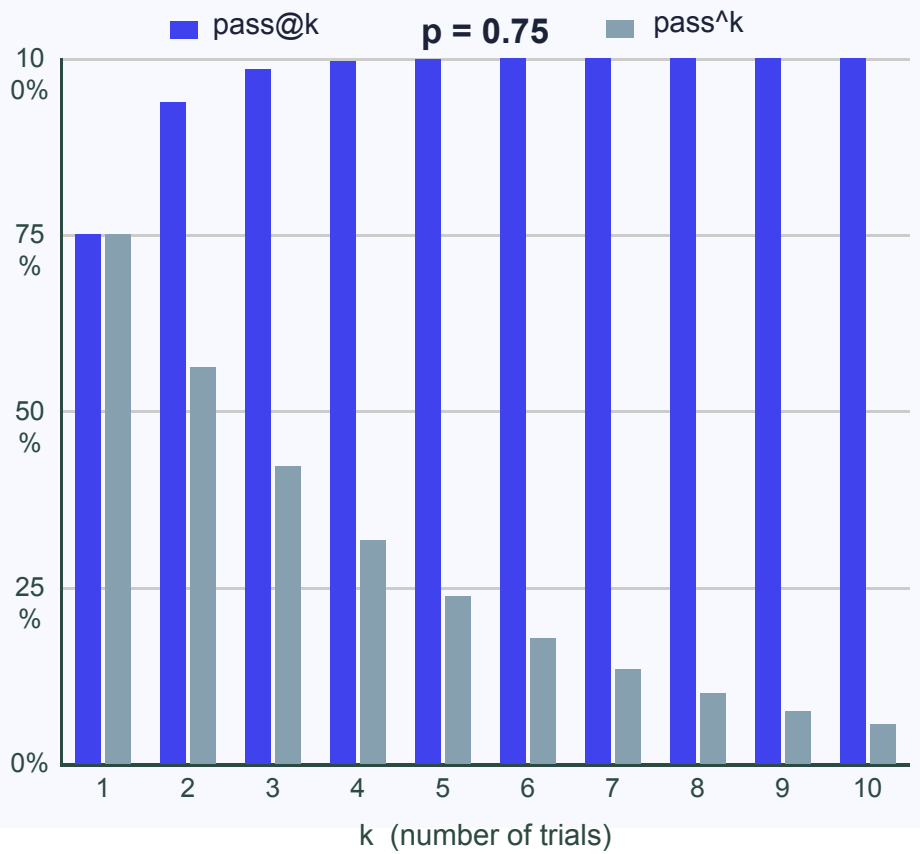
↑ grows with k · pick best of k · code gen

pass^k — ALL k trials succeed

$$\text{pass}^k = p^k$$

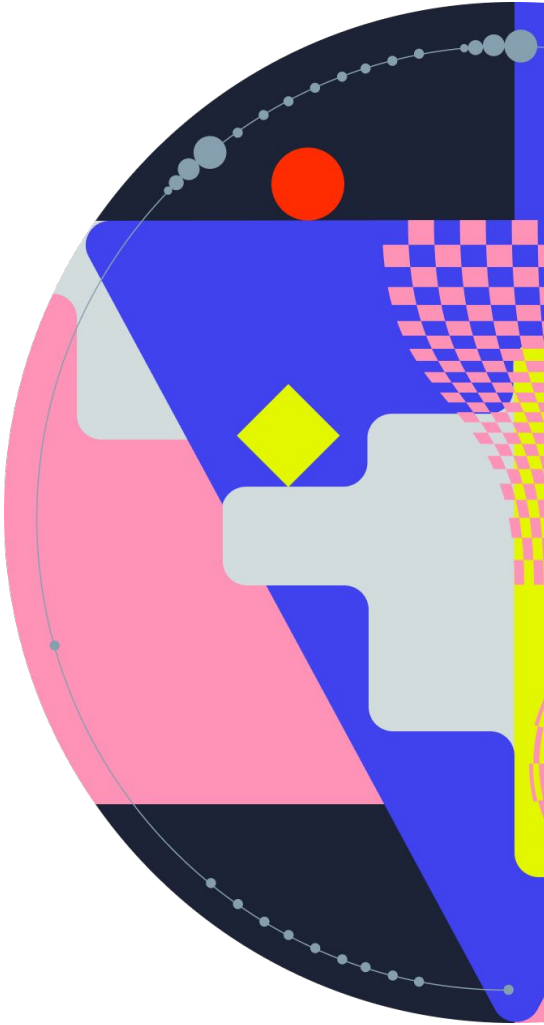
p = per-trial success rate

↓ drops with k · customer-facing · production



Grading

Part 7
Pipeline component 4 of 4
Applying criteria automatically



Grader Types — Pros & Cons

Type	Methods	✅ Strengths	⚠️ Weaknesses
Code-based	String/regex match · assert · DB state check · tool call check · transcript stats	Fast · Cheap · Objective · Reproducible · CI-friendly	Brittle to valid variations Can't capture nuance
Human	Expert review · inter-annotator agreement · A/B testing · spot-check	Gold standard · Calibrates LLM judge · Catches subtle errors	Slow · Expensive Doesn't scale to CI
LLM-as-Judge	Rubric scoring · pairwise comparison · reference-based · CoT judgment	Flexible · Captures nuance · Scales to open-ended tasks	Needs calibration ($\kappa > 0.6$) Expensive · Non-deterministic

Rule: start with code-based, add LLM judge for what code can't check, calibrate with human labels

Measuring Agreement: Cohen's κappa

$\kappa = (P_{observed} - P_{expected}) / (1 - P_{expected})$	< 0.20 Poor	0.21–0.4 Weak	0.41–0.6 Moderate	**0.61–0.8 Subtantial ✅**	0.81–1.0 Near-perfect

Corrects for chance agreement — unlike raw % agreement

Target for LLM-as-Judge: $\kappa > 0.6$ (substantial agreement with human labels)

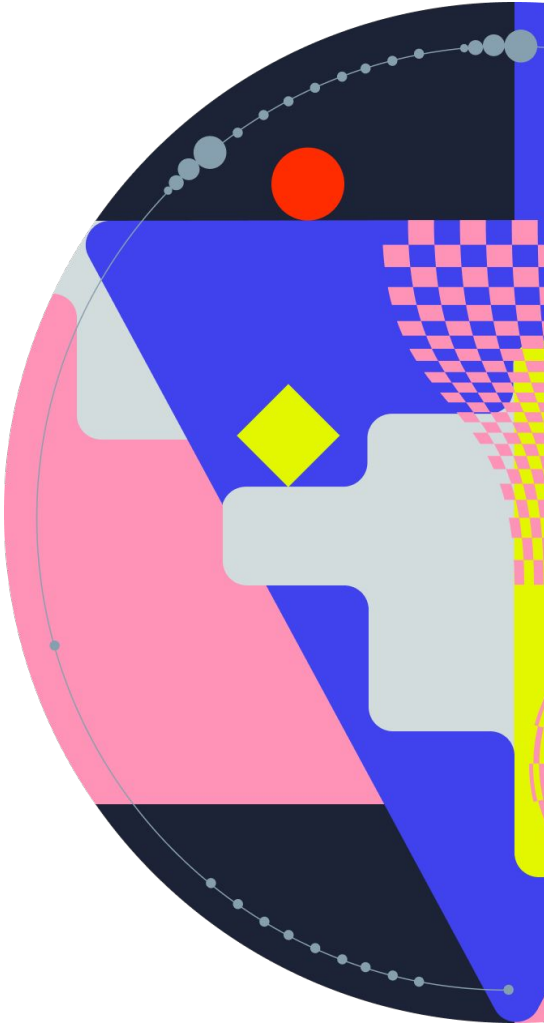
- Without human labels → no ground truth to measure judge quality
- Labeling forces you to externalize criteria (criteria drift is normal)
- Expert critiques become the most valuable few-shot examples in the judge prompt
- Periodically re-calibrate as the task suite grows

Combining Graders

One task — multiple graders — one score

- **Binary (AND)** — all graders must pass - otherwise fail
- **Weighted** — each grader gives a score, the result is a weighted sum
- **Hybrid** — critical graders are must-pass, the rest are weighted (reasoning quality).

LLM-as-Judge Deep Dive



Part 5
increase accuracy

The Judge Building Cycle

Always start with human labels — no ground truth = can't validate

- ① Manual labeling by experts: 30–100 trajectories, pass/fail + critique
- ② Human labels → judge prompt: few-shot examples, criterias, scale definitions
- ③ LLM judge runs on new tasks: automatic evaluation
- ④ Compare with human labels: Cohen's κ + confusion matrix
-
- $\kappa < 0.6$ → rework prompt → back to ③
- $\kappa \geq 0.6$ → judge ready for CI (target: substantial agreement)

Naive Judge and Its Problems

A good judge prompt needs 4 components

1. **Task context**
   Goal · inputs · constraints

2. **Agent data**
   Reasoning trace · tool calls · observations

3. **Evaluation rubric**
   What to check · scale · concrete level definitions

4. **Response format**
   JSON — so the output is parseable

Naive approach - 1 prompt, all criteria at once

```
Assess the agent's response quality:
  - usefulness  (good / so-so / bad)
  - groundedness (good / so-so / bad)
  - efficiency  (good / so-so / bad)

Reply JSON: {"usefulness": "...",
             "groundedness": "...",
             "efficiency": "..."}
```

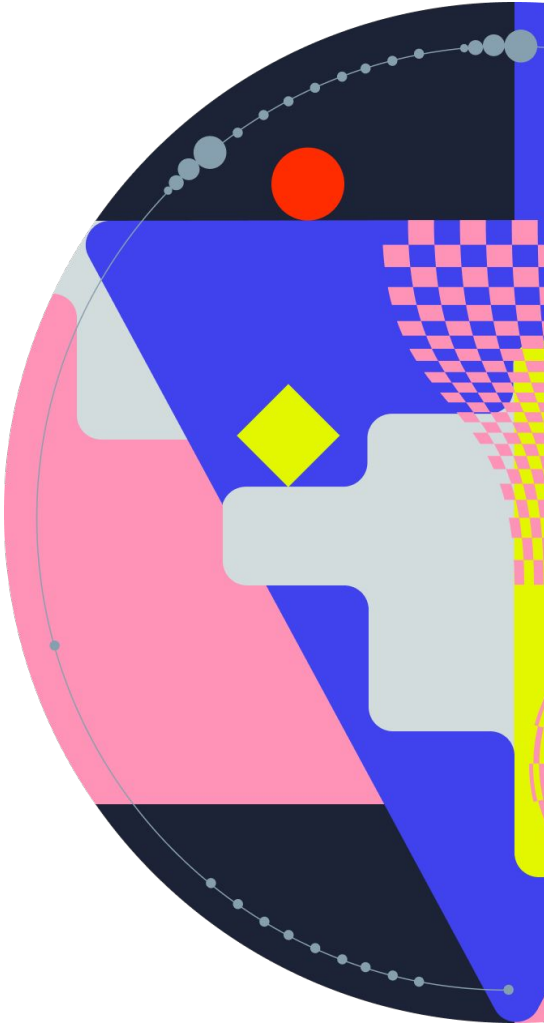
Problems this causes:

- Leniency bias — LLM marks everything 'good', misses real failures
- Halo effect — rating one criterion biases the others
- Inconsistency — same trajectory twice → different scores

5 Improvement Techniques

(1) **One criterion** = one prompt — removes halo effect, allows independent tuning
(2) **Chain-of-Thought**: reason first, then score — forces justification
(3) **Concrete rubric definitions with examples** — biggest single improvement
(4) **Escape hatch**: allow 'Unknown / Insufficient data'
(5) **Few shot examples** (golden) — include tasks with known correct answers

Eval by Agent Type



Part 6
Different agents require different grading strategies

Conversational Agents

Agent type	Multi-turn dialog + tool calls + DB state management across several conversation turns
Grader	Combo grading: State-based (DB diff) · LLM rubric judge (tone, policy compliance) · Transcript constraint (turn count)
Specifics	• Iron User required — without it multi-turn tasks cannot be completed • Grade DB state, not text (expected_goal_state vs actual_state) • Multi-dimensional: outcome + interaction quality + turn efficiency
Benchmark	τ-bench / τ2-bench (Sierra Research, 2024) — airline (50 tasks) + retail (115 tasks) pass^k at k=5 drops to 15–20% for all SOTA — agentic tasks are far harder than single-turn evals
Leaders	Qwen3-Max 82% · Claude 4.5 Sonnet ~81% Human baseline: not published

Conversational Agents — τ-bench Leaderboard

τ-bench
FROM SIERRA

Overview | Leaderboard | Visualizer | GitHub

NEW τ-bench now supports telecom domain, introduced by the τ²-bench paper.

τ-bench Leaderboard

Table | Chart | Overall | Retail | Airline | Telecom | Standard | Custom

RANK	MODEL	SUBMITTING ORG	USER SIM	PASS^1	PASS^2	PASS^3	PASS^4 ↓	AVG COST
1	Qwen3-Max-Thinking NEW	Qwen	gpt-4.1-2025-04-14	82.2%	77.1%	73.8%	71.3%	–
2	GPT-5	Sierra	gpt-4.1-2025-04-14	80.0%	73.0%	68.0%	64.0%	$0.135
3	Claude-3.7-Sonnet	Sierra	gpt-4.1-2025-04-14	61.8%	56.5%	52.9%	49.7%	–
#4	o4-mini	Sierra	gpt-4.1-2025-04-14	56.9%	48.3%	42.6%	38.0%	–
#5	GPT-4.1	Sierra	gpt-4.1-2025-04-14	54.7%	46.5%	41.4%	36.9%	–

pass^1 → pass^4 drops for ALL models. Even at pass^1 < 50% for the airline domain

Coding Agents

Agent type	Write, test, and debug code on real GitHub issues across large multi-file Python repositories
Grader	Code grader: fail-to-pass — tests that failed BEFORE the patch must pass AFTER No LLM judge needed for core evaluation — test results are objective
Specifics	• Objective grading — tests pass or fail, minimal subjectivity • A task may touch 1 to 15+ repository files • Fail-to-pass + regression: previously passing tests must not break
Benchmark	SWE-bench Verified — 500 real GitHub issues from 12 Python repos (Django, scikit-learn, sympy) Terminal-Bench — end-to-end tasks in Linux terminal (servers, scripts, networking)
Leaders	Claude 4.5 Opus 79.2% · TRAE + Doubao 78.8% · GPT-4.1 54.6% Human baseline: ~86% (expert developer)

Coding Agents — SWE-bench Verified Leaderboard

Official Leaderboards

mini-SWE-agent scores up to 74% on SWE-bench Verified in 100 lines of Python code.
Click here to learn more.

Verified | Multilingual | Lite | Full | Multimodal

Verified is a human-filtered subset of 500 instances. We use mini-SWE-agent to evaluate all models with the same harness (details).

Compare results | Agent: mini-SWE-agent v2 | Models: All models

Model	% Resolved	Avg. $	Trajs	Org	Date	Agent
Claude 4.5 Opus (high reasoning)	76.80	$0.75			2026-02-17	2.0.0
Gemini 3 Flash (high reasoning)	75.80	$0.36			2026-02-17	2.0.0
MiniMax M2.5 (high reasoning)	75.80	$0.07			2026-02-17	2.0.0
Claude Opus 4.6	75.60	$0.55			2026-02-17	2.0.0
GPT-5-2 Codex	72.80	$0.45			2026-02-19	2.0.0
GLM-5 (high reasoning)	72.80	$0.53			2026-02-17	2.0.0
GPT-5-2 (high reasoning)	72.80	$0.47			2026-02-17	2.0.0

~4% (GPT-4, Jan 2024) → 77% (Claude 4.5 Opus, 2025). Fastest-improving benchmark in AI.

Research Agents

Agent type	Web search, information synthesis, report generation and multi-step factual Q&A
Grader	Exact match for factual tasks (BrowseComp) · LLM judge: groundedness + coverage + source quality Human calibration required — subjectivity is higher than any other agent type
Specifics	• 'comprehensive', 'well-sourced' depend on context — most subjective agent type • BrowseComp tasks can't be solved in one query — chained searches + cross-referencing required • Human baseline is low — even experts with internet access fail many tasks
Benchmark	BrowseComp (OpenAI, 2025) — hard factual questions, exact match, deliberately non-templated GAIA (Meta, 2023) — ~450 tasks, 3 difficulty levels, multi-step reasoning + tool use
Leaders	BrowseComp: deep research models ~51% · Human baseline: ~5–10% (very hard!) GAIA: multi-model ensembles ~75% · Human baseline: ~92%

Research Agents — GAIA Leaderboard

Leaderboard

Submission made by our team are labelled "GAIA authors". While we report average scores over different runs when possible in our paper, we only report the best run in the leaderboard.

See below for submissions.

Citation

Results: Test

Agent name	Model family	organisation	Average score (…	Level 1 sco
openJiuwen-deepagent	GPT5, Gemini-3-Pro	openJiuwen	91.69	98.92
Lemon	GPT5, Gemini-3-pro, o3	LR AILab of Lenovo CTO Org	91.36	96.77
JoinAI_V2.2	GPT 5, Gemini 3 Pro, DeepSeek 3.1, Qwen 3	JoinAI-CMCC	90.7	98.92
Nemotron-ToolOrchestra-0106	Nemotron-ToolOrchestrator-8B, GPT-5, Claude Opus 4.1, Qwen2.5-Math-72B	NVIDIA	90.37	96.77
Nemotron-ToolOrchestra-0107	Nemotron-ToolOrchestrator-8B, GPT-5, Claude Opus 4.1, Qwen2.5-Math-72B	NVIDIA	90.37	96.77
SU Zero - Shuqian Series Pro MAX	Self Consistency 35	Suzhou AI Lab, Shuqian Tech	90.03	98.92
JoinAI_V2.1	GPT, Gemini, DeepSeek, Qwen	JoinAI-CMCC	90.03	98.92
HALO V1217-1		Microsoft AI Asia - Ads	89.37	96.77
ShawnAgent_v3.1	GPT5.2, Claude Sonnet 4.5, Gemini 3 Pro		89.37	96.77
Su Zero + SO Pro	GPT, Gemini, Claude	Suzhou AI Lab, Shuqian Tech	89.04	97.85
JoinAI_V2	GPT, Gemini, DeepSeek, Qwen	JoinAI-CMCC	89.04	98.92
ShawnAgent_v3.0	GPT5.2, Claude Sonnet 4.5, Gemini 3 Pro		89.04	96.77

Refresh

Human baseline ~92% on GAIA. BrowseComp human baseline: only ~5–10% — intentionally hard.

Research Agents

Agent type	GUI agents — control browser and OS via screenshots, mouse clicks, and keyboard input
Grader	Execution-based: programmatic validators check URL, page state, or VM file system No LLM judge needed — final system state is objective
Specifics	• Requires real or sandboxed GUI environment (VM with snapshot/restore before each run) • Strong tool-calling agent ≠ strong GUI agent: different interfaces and different skills • Agents take 1.4–2.7× more steps than humans and ~10× more time
Benchmark	WebArena (CMU, 2023) — 812 tasks on self-hosted websites, cross-site reasoning (5–15 steps) OSWorld (HKU, 2024) — 369 tasks in real OSes (Ubuntu, Windows, macOS), 134 eval functions
Leaders	WebArena: DeepSeek v3.2 74.3% · OpAgent 71.6% · Human ~78% OSWorld: Kimi K2.5 63.3% · Claude Sonnet 4.5 62.9% · Human 72.4%

Computer Use Agents — Leaderboards

X-WebArena-Leaderboard

Файл Правка Вид Вставка Формат Данные Инструменты Расширения Справка

Меню 100% Только комментирование

A1 a

	A	B	C	D	E	F	G
1	a	Open?	Model Size (billion)	Model	Success Rate (%)	Result Source	Work
2	02/2026	✔	-	Deepseek v3.2	74.3	WebTactix	WebTactix
3	01/2026	✔	-	OpAgent	71.6	OpAgent	OpAgent
4	12/2025	✔	-	ColorBrowserAgent	71.2	ColorBrowserAgent	ColorBrowserAgent
5	10/2025	✔	-	Claude Code + GBOX MCP	68	GBOX AI	GBOX AI
6	09/2025	✗	-	DeepSky Agent	66.9	Self-reported	DeepSky Agent
7	10/2025	✗		Narada AI	64.2	Self-reported	Narada AI
8	02/2025	✔	-	IBM CUGA	61.7	IBM CUGA	IBM CUGA
9	01/2025	✗	-	OpenAI Operator	58.1	OpenAI CUA	OpenAI CUA
10	08/2024	✗	-	Jace.AI	57.1	Reported by zetalabs.ai	https://www.jace.ai/
11	12/2025	✔	-	WebOperator + GPT-4o	54.6	WebOperator	WebOperator
12	12/2024	✗	-	ScribeAgent + GPT-4o	53	ScribeAgent	ScribeAgent
13	01/2025	✔	-	AgentSymbiotic	52.1	AgentSymbiotic	AgentSymbiotic
14	01/2025	✔	-	Learn-by-Interact	48	Learn-by-interact	Learn-by-interact
15	10/2024	✔	-	AgentOccam-Judge	45.7	AgentOccam-Judge	AgentOccam-Judge
16	08/2024	✗	-	WebPilot	37.2	WebPilot	WebPilot
17	10/2024	✔	-	GUI-API Hybrid Agent	35.8	Beyond Browsing	Beyond Browsing
18	09/2024	✔	-	Agent Workflow Memory	35.5	AWM	AWM
19	04/2024	✔	-	SteP	33.5	SteP	SteP
20	06/2025	✔	12	TTI	26.1	TTI	TTI
21	04/2024	✔	-	BrowserGym + GPT-4	23.5	WorkArena	BrowserGym
22	01/2025	✔	32	AgentTrek-1.0-32B	22.4	AgentTrek	AgentTrek
23	04/2024	✔	-	GPT-4 + Auto Eval	20.2	Auto Eval & Refine	Auto Eval & Refine
24	06/2024	✔	-	GPT-4o + Tree Search	19.2	Tree Search for LM Agents	Tree Search for LM Agents
25	04/2024	✔	7	AutoWebGLM	18.2	AutoWebGLM	AutoWebGLM
26	01/2025	✔	8	NNetNav	16.3	NNetscape	NNetscape
27	06/2023	✔	-	gpt-4-0613	14.9	WebArena	GPT
28	05/2024	✔	-	gpt-4o-2024-05-13	13.1	WebArena Team	GPT
29	06/2023	✔	-	gpt-4-0613	11.7	WebArena	GPT
30	05/2024	✔	72	Patel et al + 2024	9.36	Patel et al + 2024	Patel et al + 2024
31	03/2023	✔	-	gpt-3.5-turbo-16k-0613	8.87	WebArena	GPT
32	09/2023	✔	72	Qwen-1.5-chat-72b	7.14	Patel et al + 2024	Qwen
33	12/2023	✔	-	Gemini Pro	7.12	WebArena	Gemini Pro
34	04/2024	✔	70	Llama3-chat-70b	7.02	WebArena Team	Llama3
35	10/2024	✔	7	Synatra-CodeLLama7b	6.28	Synatra	Synatra
36	10/2023	✔	70	Lemur-chat-70b	5.3	Lemur	Lemur
37	03/2024	✔	7	Agent Flan	4.68	Agent Flan	Agent Flan

1 WebArena VisualWebArena

OSWorld-Verified Results

Foundation E2E GUI All

Filters:

Model Type: All ▼ Max Steps: All Clear Filters

Sort by: Score (Highest first)

Rank	Model & Date	Approach & Details	Success Rate (Avg±Std)
1	Kimi K2.5 *Moonshot AI* Jan 31, 2026 Kimi Team	**Type:** General model **Max Steps:** 100 **Runs:** 1	63.3%
2	claude-sonnet-4-5-20250929 *Anthropic* Oct 31, 2025 Anthropic, '25	**Type:** General model **Max Steps:** 100 **Runs:** 1	62.9%
3	Seed-1.8 *ByteDance Seed* Dec 18, 2025 ByteDance Seed	**Type:** General model **Max Steps:** 100 **Runs:** 1	61.9%
4	claude-sonnet-4-5-20250929 *Anthropic* Oct 31, 2025 Anthropic, '25	**Type:** General model **Max Steps:** 50 **Runs:** 1	58.1%
5	EvoCUA-20260105 *Meituan LongCat Team* Jan 6, 2026	**Type:** General model **Max Steps:** 50 **Runs:** 1	56.7%

Specialized browser agents outperform general models on WebArena. OSWorld gap to human is closing.

Operational Metrics — Not Just Quality

Production readiness requires efficiency tracking alongside pass rates

- **Latency** — task execution time p50/p95: compare across agent versions
- **Cost** — API calls + compute: eval can become more expensive than production
- **Error rate** — timeouts, exceptions, tool failures: must be below threshold

Moving to Practice

Now let's see how it all works in the notebook

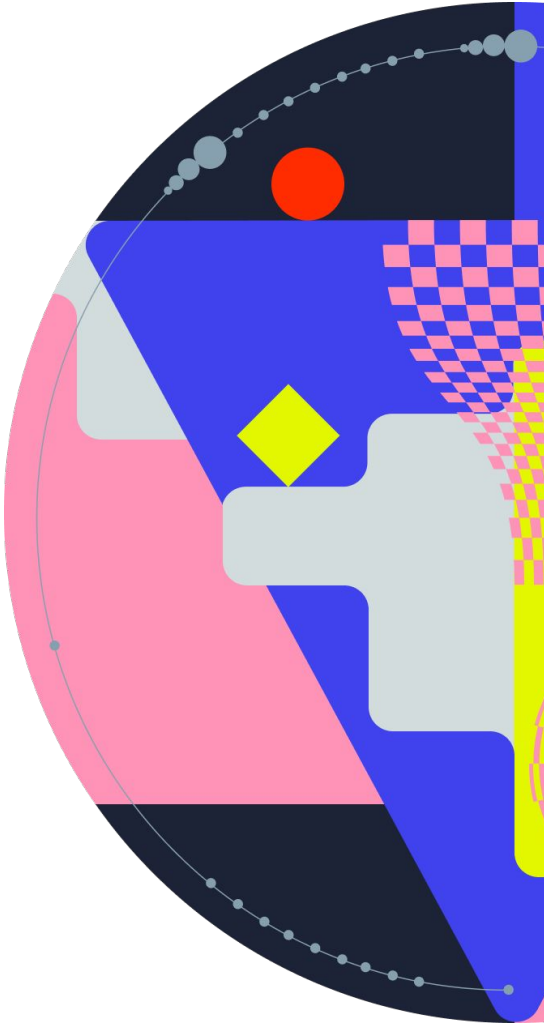
1. **Airline agent + task suite**
   Agent structure · 12-task suite · balance visualization

2. **Code-based graders**
   state_grader · policy_grader · run on 3 tasks

3. **Iron User v1 → v2**
   Infinite loop (77 steps) → stable run (8 steps)

4. **LLM-as-Judge iterations**
   v1 (κ 0.25) → v4 (κ 0.72) — live calibration

Wrap-Up



Part 7
Checklist and key takeaways

Key Takeaways

1 **Eval = 4 components**
Task Suite · Infrastructure · Criteria · Grading
Weakness in any one devalues all others

2 **Start early, not perfectly**
20 tasks from real problems
> 0 tasks 'eventually'

3 **Eval is a loop, not a checkpoint**
Results → new tasks,
improved criteria, better graders

4 **Grade the outcome, not the path**
Agents find unexpected valid solutions.
Checking step sequences is brittle

5 **LLM-as-Judge requires discipline**
Per-criterion prompts + CoT + calibration
on human labels — the required minimum

6 **Stochasticity is a property, not a bug**
pass@k / pass^k and multiple trials
let you account for it

7 **Capability vs regression evals**
Former drives progress,
latter prevents degradation

Conclusion: From Eval to a Quality Culture

Flexibility by design

Put params (temperature, prompts, models) in configs from the start — experiments become routine

Quality goals before optimization

'90% accuracy for category X at latency <5s' protects against endless iteration

2–5 metrics, not twenty

80/20 rule: fewer but reliable. Add new ones when existing ones stop giving signal

Save every run

Agent version, params, date, per-task metrics — into a DB or file. Any run retrievable a month later

Tails matter more than averages

Analyze worst and best cases after each run. Every failure mode is a potential new eval task

Stability over average

Standard deviation is as important a signal as the mean. High variance = warning sign

Thresholds protect

'accuracy ≥ 85%', 'latency ≤ 3s' — no-go, even if the aggregate score formally improved

Quality first, then optimize

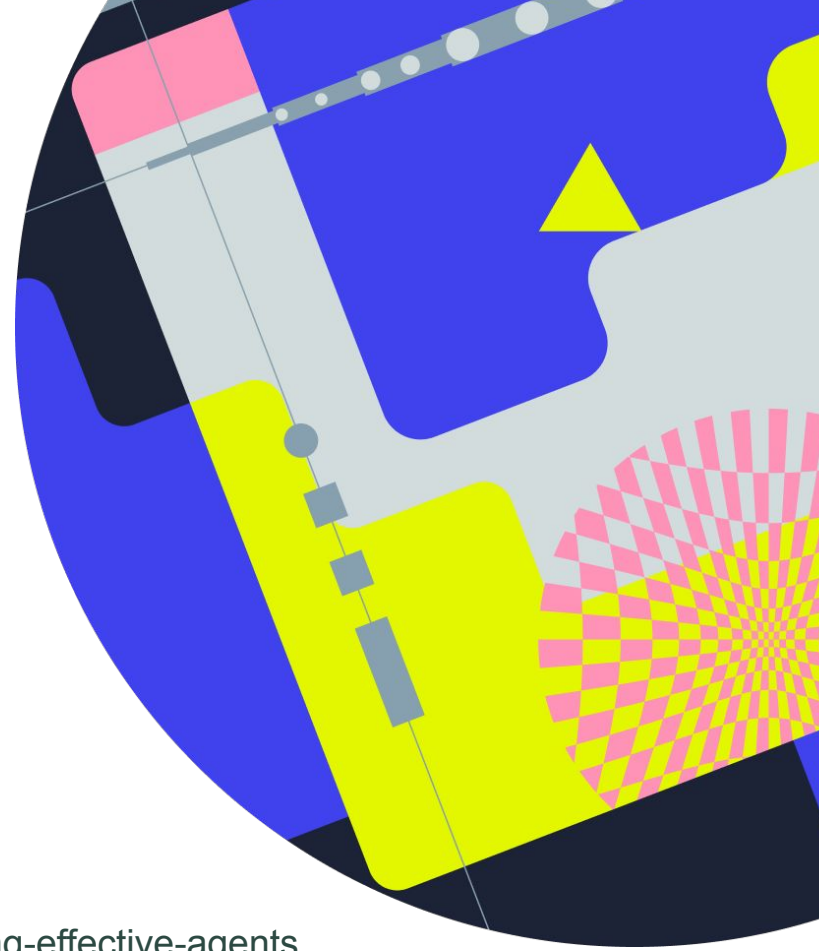
Start with the strongest model and full context. Lock in the quality ceiling — then optimize cost

The loop closes through production

User logs and feedback → new eval tasks. Online and offline metrics must correlate

Eval is part of engineering culture. Teams that invest early in eval infrastructure ship AI products faster and with far more confidence.

Fin



References:

· τ-bench · taubench.com (Yao et al., Sierra, 2024)
· SWE-bench · swebench.com (Princeton, 2024)
· WebArena · webarena.dev (CMU, 2023)
· OSWorld · os-world.github.io (HKU, 2024)
· LLM Judge calibration · hamel.dev/blog/posts/llm-judge
· Building effective agents · anthropic.com/engineering/building-effective-agents